

AVIATION ANALYSIS PROJECT

Assessing Aircraft Accident Trends and Safety Risks

Project Overview

This project analyzes **aircraft accident data (1962-2023)** to identify trends, risk factors, and insights into aviation safety. Using **Python (pandas, NumPy, Matplotlib)** and **Tableau**, we aim to:

-  **Clean and preprocess** aviation accident data.
-  **Explore trends** in accident frequency, causes, and severity.
-  **Visualize key insights** through charts and interactive dashboards.
-  **Identify risk factors** that contribute to aviation incidents.

Dataset Information

- **'AviationData.csv'** – Contains accident reports, aircraft details, causes, and locations.
- **'USState_Codes.csv'** – Mapping of state codes to full state names for analysis.

Key Questions

- What are the most common causes of aviation accidents?
- How have accident trends changed over time?
- Which aircraft types and flight phases have the highest risk?
- What role does weather or location play in accidents?

Let's dive into the data!



Data Loading

Data Loading: Importing the Dataset for Analysis

The first step in any data analysis process is loading the dataset into the environment. This allows us to inspect, clean, and prepare the data for further exploration.

1. Importing Necessary Libraries

Before loading the datasets provided, essential **Python libraries** are imported for data manipulation and analysis.

2. Reading the Dataset

The datasets that are provided will then be loaded using **Pandas**.

3. Inspecting the Data

After loading, initial checks shall then be performed to understand the dataset structure

4. Verifying Data Integrity

To ensure data integrity, the following are checked for:

- **Duplicate values** – using `.duplicated().sum()`
- **Inconsistent column names** – using `aviation_df.columns`
- **Missing values** – using `.isnull().sum()`

```
In [71]: ▶ # Import necessary libraries
```

```
import pandas as pd # For data manipulation
import numpy as np  # For numerical operations
import matplotlib.pyplot as plt # For data visualization
import seaborn as sns # For statistical plots
```

```
In [72]: ▶ # Loading the datasets as DataFrames
```

```
aviation_df = pd.read_csv("AviationData.csv", encoding="latin1", low_memory=False)
state_codes_df = pd.read_csv("USState_Codes.csv", encoding="latin1")
```

Understanding the DataFrame

The dataframe contains **aviation accident records** from 1962 to 2023. Each row represents an individual accident, including details such as:

- **Accident Number** – A unique identifier for each incident.
- **Date** – When the accident occurred.
- **Location** – City and state of the accident.
- **Aircraft Type** – The type of aircraft involved.
- **Injuries and Fatalities** – Number of people affected.
- **Cause of Accident** – Probable causes based on investigation reports.

Additionally, we have a **USState_Codes.csv** file that maps **state abbreviations to full names** for location-based analysis.

Let's start by examining the dataframe structure!

In [73]: `aviation_df.head()`

Out[73]:

	Event.Id	Investigation.Type	Accident.Number	Event.Date	Location	Country
0	20001218X45444	Accident	SEA87LA080	1948-10-24	MOOSE CREEK, ID	United States
1	20001218X45447	Accident	LAX94LA336	1962-07-19	BRIDGEPORT, CA	United States
2	20061025X01555	Accident	NYC07LA005	1974-08-30	Saltville, VA	United States
3	20001218X45448	Accident	LAX96LA321	1977-06-19	EUREKA, CA	United States
4	20041105X01764	Accident	CHI79FA064	1979-08-02	Canton, OH	United States

5 rows × 31 columns

In [74]: `aviation_df.shape`

Out[74]: (88889, 31)

In [75]: `aviation_df.columns`

Out[75]: Index(['Event.Id', 'Investigation.Type', 'Accident.Number', 'Event.Date',
'Location', 'Country', 'Latitude', 'Longitude', 'Airport.Code',
'Airport.Name', 'Injury.Severity', 'Aircraft.damage',
'Aircraft.Category', 'Registration.Number', 'Make', 'Model',
'Amateur.Built', 'Number.of.Engines', 'Engine.Type', 'FAR.Description',
'Schedule', 'Purpose.of.flight', 'Air.carrier', 'Total.Fatal.Injuries',
'Total.Serious.Injuries', 'Total.Minor.Injuries', 'Total.Uninjured',
'Weather.Condition', 'Broad.phase.of.flight', 'Report.Status',
'Publication.Date'],
dtype='object')

In [76]: `# Let's look at the concise summary of the data`
`aviation_df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 88889 entries, 0 to 88888
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Event.Id                             88889 non-null  object
1   Investigation.Type                   88889 non-null  object
2   Accident.Number                     88889 non-null  object
3   Event.Date                          88889 non-null  object
4   Location                            88837 non-null  object
5   Country                             88663 non-null  object
6   Latitude                           34382 non-null  object
7   Longitude                           34373 non-null  object
8   Airport.Code                       50249 non-null  object
9   Airport.Name                       52790 non-null  object
10  Injury.Severity                     87889 non-null  object
11  Aircraft.damage                     85695 non-null  object
12  Aircraft.Category                   32287 non-null  object
13  Registration.Number                 87572 non-null  object
14  Make                               88826 non-null  object
15  Model                              88797 non-null  object
16  Amateur.Built                      88787 non-null  object
17  Number.of.Engines                   82805 non-null  float64
18  Engine.Type                         81812 non-null  object
19  FAR.Description                     32023 non-null  object
20  Schedule                           12582 non-null  object
21  Purpose.of.flight                  82697 non-null  object
22  Air.carrier                        16648 non-null  object
23  Total.Fatal.Injuries                77488 non-null  float64
24  Total.Serious.Injuries              76379 non-null  float64
25  Total.Minor.Injuries                76956 non-null  float64
26  Total.Uninjured                     82977 non-null  float64
27  Weather.Condition                   84397 non-null  object
28  Broad.phase.of.flight               61724 non-null  object
29  Report.Status                       82508 non-null  object
30  Publication.Date                    75118 non-null  object
dtypes: float64(5), object(26)
memory usage: 21.0+ MB
```

```
In [77]: aviation_df.describe()
```

```
Out[77]:
```

	Number.of.Engines	Total.Fatal.Injuries	Total.Serious.Injuries	Total.Minor.Injuries	Tota
count	82805.000000	77488.000000	76379.000000	76956.000000	82
mean	1.146585	0.647855	0.279881	0.357061	
std	0.446510	5.485960	1.544084	2.235625	
min	0.000000	0.000000	0.000000	0.000000	
25%	1.000000	0.000000	0.000000	0.000000	
50%	1.000000	0.000000	0.000000	0.000000	
75%	1.000000	0.000000	0.000000	0.000000	
max	8.000000	349.000000	161.000000	380.000000	

DataFrame Summary

The `.info()` method gives us an overview of the dataframe, including:

- **Total entries:** Number of rows in the dataframe.
- **Column names & data types:** Helps us understand which columns are numerical, categorical, or text-based.
- **Missing values:** Identifies columns with missing data that may require cleaning.

Key Observations

1. The data has **88889 rows** and **31 columns**.
2. There are 5 columns that have **Numeric Data(floats)** and 26 columns that have **Categorical Data(objects)**.
3. 29 columns out of the 31 have **missing values**.

Statistical Summary of the DataFrame

The `.describe()` function provides key summary statistics for **numerical columns**, helping us understand:

- **Count** – Number of non-missing values.
- **Mean** – Average value for each column.
- **Standard Deviation** – How spread out the values are.
- **Min & Max** – The range of values.
- **Quartiles (25%, 50%, 75%)** – Helps identify distribution patterns.

Key Insights

1. Identify **outliers** by looking at min/max values.
2. Check for **skewed distributions** using mean vs. median (50%).

```
In [78]: ► aviation_df.duplicated().sum()
```

```
Out[78]: 0
```

```
In [79]: ► #Identify missing values  
aviation_df.isnull().sum()
```

```
Out[79]: Event.Id                0  
Investigation.Type              0  
Accident.Number                0  
Event.Date                     0  
Location                       52  
Country                        226  
Latitude                       54507  
Longitude                      54516  
Airport.Code                   38640  
Airport.Name                   36099  
Injury.Severity                1000  
Aircraft.damage                3194  
Aircraft.Category              56602  
Registration.Number            1317  
Make                           63  
Model                          92  
Amateur.Built                  102  
Number.of.Engines              6084  
Engine.Type                    7077  
FAR.Description                56866  
Schedule                       76307  
Purpose.of.flight              6192  
Air.carrier                    72241  
Total.Fatal.Injuries           11401  
Total.Serious.Injuries         12510  
Total.Minor.Injuries           11933  
Total.Uninjured                5912  
Weather.Condition              4492  
Broad.phase.of.flight          27165  
Report.Status                  6381  
Publication.Date               13771  
dtype: int64
```

Data Cleaning

Preparing the Dataset for Analysis

Data cleaning is a crucial step in the data analysis process. It ensures that our dataset is accurate, consistent, and ready for further analysis. Below are the key steps performed to clean our aviation accident dataset:

1. Standardizing Column Names

To ensure consistency in the dataset, the following was done:

- Capitalizing the first letter of every word in column names.

2. Handling Missing Values

Missing values can affect the reliability of our analysis. Different strategies were used to handle them:

- **Numerical Data:** Filled missing values with the median, rounding up to the nearest whole number.
- **Categorical Data:** Filled missing values with the most frequent value (mode).
- **Columns with High Missing Percentages:** Considered removing features with excessive missing data that provided little value.

3. Adding New Features for Better Insights

New features were added to enhance our analysis:

Outcome: A Clean & Structured Dataset

After these steps, the dataset is now:

- Free of missing values.
- Consistent in format.
- Enhanced with new features for deeper analysis.

```
In [80]: ▶ # Standardizing column names: Replace dots with underscores and capitalize
aviation_df.columns = aviation_df.columns.str.replace('.', '_').str.title()

# Display updated column names
aviation_df.columns
```

```
Out[80]: Index(['Event_Id', 'Investigation_Type', 'Accident_Number', 'Event_Date',
               'Location', 'Country', 'Latitude', 'Longitude', 'Airport_Code',
               'Airport_Name', 'Injury_Severity', 'Aircraft_Damage',
               'Aircraft_Category', 'Registration_Number', 'Make', 'Model',
               'Amateur_Built', 'Number_Of_Engines', 'Engine_Type', 'Far_Description',
               'Schedule', 'Purpose_Of_Flight', 'Air_Carrier', 'Total_Fatal_Injuries',
               'Total_Serious_Injuries', 'Total_Minor_Injuries', 'Total_Uninjured',
               'Weather_Condition', 'Broad_Phase_Of_Flight', 'Report_Status',
               'Publication_Date'],
              dtype='object')
```

This ensures that the column names have the same format.

Handling Missing Data

Missing values can impact analysis and visualization accuracy. We need to:

1. **Identify columns with missing values** – Check how many and where they appear.

2. **Decide on a handling strategy:**

- **Fill missing values** using appropriate methods (e.g., mean, median, mode).
- **Drop columns or rows** if missing values are excessive.
- **Use domain knowledge** to make informed replacements.

Let's first analyze the extent of missing data before deciding the best approach!

```
In [81]: ▶ # Checking for missing values in the dataset
missing_values = aviation_df.isnull().sum()

# Displaying only columns with missing values
missing_values = missing_values[missing_values > 0]

# Checking percentage of missing values
missing_percentage = (missing_values / len(aviation_df)) * 100

# Display missing values with percentage
missing_data = pd.DataFrame({'Missing Values': missing_values, 'Percentage': missing_percentage})
```

Out[81]:

	Missing Values	Percentage
Location	52	0.058500
Country	226	0.254250
Latitude	54507	61.320298
Longitude	54516	61.330423
Airport_Code	38640	43.469946
Airport_Name	36099	40.611324
Injury_Severity	1000	1.124999
Aircraft_Damage	3194	3.593246
Aircraft_Category	56602	63.677170
Registration_Number	1317	1.481623
Make	63	0.070875
Model	92	0.103500
Amateur_Built	102	0.114750
Number_Of_Engines	6084	6.844491
Engine_Type	7077	7.961615
Far_Description	56866	63.974170
Schedule	76307	85.845268
Purpose_Of_Flight	6192	6.965991
Air_Carrier	72241	81.271023
Total_Fatal_Injuries	11401	12.826109
Total_Serious_Injuries	12510	14.073732
Total_Minor_Injuries	11933	13.424608
Total_Uninjured	5912	6.650992
Weather_Condition	4492	5.053494
Broad_Phase_Of_Flight	27165	30.560587
Report_Status	6381	7.178616
Publication_Date	13771	15.492356

Dropping Columns or Rows

- If a column has too many missing values (e.g., above 50-60%), we may drop it.
- If only a few rows are missing values in critical columns, we can drop them.

```
In [82]: ▶ # Drop columns with more than 60% missing values
aviation_df = aviation_df.dropna(thresh=len(aviation_df) * 0.4, axis=1)

# Drop rows with any missing critical values
aviation_df = aviation_df.dropna(subset=['Make', 'Model', 'Amateur_Built'])
```

Filling Missing Values

- For numerical columns, we can fill missing values with mean, median, or mode.
- For categorical columns, we can fill with mode (most common value) or 'Unknown'.

```
In [83]: ▶ # Filling numerical columns with median
aviation_df['Total_Fatal_Injuries'] = aviation_df['Total_Fatal_Injuries'].
aviation_df['Total_Serious_Injuries'] = aviation_df['Total_Serious_Injuries'].
aviation_df['Total_Minor_Injuries'] = aviation_df['Total_Minor_Injuries'].
aviation_df['Total_Uninjured'] = aviation_df['Total_Uninjured'].fillna(rou
aviation_df['Number_Of_Engines'] = aviation_df['Number_Of_Engines'].fillna
```

```
In [84]: # Selecting only categorical columns
categorical_cols = aviation_df.select_dtypes(include=['object'])

# Checking for missing values
missing_values_cat = categorical_cols.isnull().sum()

# Displaying only columns with missing values
missing_values_cat = missing_values_cat[missing_values_cat > 0]

# Checking percentage of missing values
missing_percentage_cat = (missing_values_cat / len(aviation_df)) * 100

# Display missing values with percentage
missing_data_cat = pd.DataFrame({'Missing Values': missing_values_cat, 'Percentage': missing_percentage_cat})
missing_data_cat
```

Out[84]:

	Missing Values	Percentage
Location	48	0.054128
Country	225	0.253727
Airport_Code	38474	43.386184
Airport_Name	35942	40.530910
Injury_Severity	979	1.103994
Aircraft_Damage	3154	3.556688
Registration_Number	1208	1.362232
Engine_Type	6928	7.812535
Purpose_Of_Flight	6073	6.848373
Weather_Condition	4381	4.940346
Broad_Phase_Of_Flight	27013	30.461896
Report_Status	6335	7.143824
Publication_Date	13754	15.510048

Approach

Categorize missing values into three levels:

1. Low Missing Percentage ($\leq 5\%$)

- Filled using **Mode** (most frequent value in the column).
- Ensures minimal data distortion.

2. Moderate Missing Percentage (5% - 20%)

- Replaced with "Unknown" or "Not Available" .
- Helps preserve data integrity when missing values may hold meaning.

3. High Missing Percentage ($> 20\%$)

- Uses **Forward Fill (ffill)** first, filling missing values with the previous row's value.
- If values are still missing, applies **Backward Fill (bfill)** to fill using the next row's value.

```
In [85]: total_rows = len(aviation_df)

for col in missing_values_cat.index:
    missing_percentage_cat = (missing_values_cat[col] / total_rows) * 100

    if missing_percentage_cat <= 5: # If missing values are 5% or less
        aviation_df[col] = aviation_df[col].fillna(aviation_df[col].mode())

    elif 5 < missing_percentage_cat <= 20: # If missing values are between 5% and 20%
        aviation_df[col] = aviation_df[col].fillna("Unknown")

    else: # If missing values are more than 20%
        aviation_df[col] = aviation_df[col].fillna(method='ffill') # First fill with forward fill
        aviation_df[col] = aviation_df[col].fillna(method='bfill') # Then fill with backward fill

# Confirm missing values are handled
print("Missing values after handling:\n", aviation_df.isnull().sum())
```

Missing values after handling:

Event_Id	0
Investigation_Type	0
Accident_Number	0
Event_Date	0
Location	0
Country	0
Airport_Code	0
Airport_Name	0
Injury_Severity	0
Aircraft_Damage	0
Registration_Number	0
Make	0
Model	0
Amateur_Built	0
Number_Of_Engines	0
Engine_Type	0
Purpose_Of_Flight	0
Total_Fatal_Injuries	0
Total_Serious_Injuries	0
Total_Minor_Injuries	0
Total_Uninjured	0
Weather_Condition	0
Broad_Phase_Of_Flight	0
Report_Status	0
Publication_Date	0
dtype: int64	

Cleaning Engine_Type Column in aviation_df

The `aviation_df['Engine_Type']` column contains different variations of "Unknown" and "None" that need to be standardized.

```
In [86]: ► aviation_df['Engine_Type'] = aviation_df['Engine_Type'].replace(
        [ 'Unknown', 'UNK'], 'Unknown'
    ).replace(
        [ 'None', 'NONE'], 'None'
    )

    aviation_df['Engine_Type'].unique()
```

```
Out[86]: array(['Reciprocating', 'Unknown', 'Turbo Fan', 'Turbo Shaft',
                'Turbo Prop', 'Turbo Jet', 'None', 'Electric', 'Hybrid Rocket',
                'Geared Turbofan', 'LR'], dtype=object)
```

Now that the data is filled with no none or null values we can proceed with converting the data in the `Event_Date` column to the correct format.

```
In [87]: ► # Convert Event_Date to datetime format
    aviation_df['Event_Date'] = pd.to_datetime(aviation_df['Event_Date'])
```

Extracting Time-Based Features

Since accidents often have seasonal trends, let's create features from the date column:

```
In [88]: ► # Extracting key time-related features
    aviation_df['Year'] = aviation_df['Event_Date'].dt.year
    aviation_df['Month'] = aviation_df['Event_Date'].dt.month
    aviation_df['Day_Of_Week'] = aviation_df['Event_Date'].dt.day_name()
```

The above will:

- Help to analyze if accidents occur more frequently on weekends.
- Detect the seasonal trends (e.g., winter vs. summer accidents).

Counting Incidents

To better understand aviation accident trends, create new features that count the number of incidents based on different categories.

```
In [89]: ▶ aviation_df['Manufacturer_Incident_Count'] = aviation_df.groupby('Make')['Model_Incident_Count'] = aviation_df.groupby('Model')['Flight_Purpose_Incident_Count'] = aviation_df.groupby('Purpose_of_Flight')['Weather_Incident_Count'] = aviation_df.groupby('Weather_Condition')
```

groupby('Make') groups the dataset by `Make` :

- Counts how many times each manufacturer appears in the dataset (i.e., how many accidents involve aircraft from that manufacturer).
- Assigns the count as a new column `manufacturer_incident_count` for each row.

groupby('Model') groups the dataset by `Model` :

- Counts how many accidents occurred for each model.
- Stores the count in `model_incident_count`.

groupby('Purpose_Of_Flight') groups accidents based on the `Purpose_Of_Flight` :

- Counts how many accidents occurred for each type of flight.
- Creates `flight_purpose_incident_count` to store the count.

groupby('Weather_Condition') groups accidents by `Weather_Condition` :

- Counts how many accidents happened under each weather type.
- Stores this in `weather_incident_count` to help analyze how weather impacts aviation safety.

By counting the number of incidents based on different categories, we are able to:

- Identify high-risk aircraft manufacturers and models.
- Detect which types of flights have more accidents.
- Analyze weather conditions associated with aviation incidents.
- Improve accident prediction models by adding meaningful features.

Identify Key Features

Categorize the features into potentially useful and less useful based on their relevance to accident analysis.

The potentially useful features are to be kept and the less useful features are to be dropped.

Potentially useful features to be kept:

- `Event_Date` : Helps analyze accident trends over time.
- `Airport_Name` : Identifying high-risk airports.
- `Contry` and `Location` : Geographical distribution of the accidents.
- `Injury_Severity` : The severity of accidents.
- `Aircraft_Damage` : Understanding the extent of damage.

- Make and Model : Aircraft manufacturers & models with the most accidents.
 - Number_Of_Engines and Engine_Type : Engine configuration influence on accidents.
 - Purpose_Of_Flight : Comparing accidents between commercial & private flights.
 - Total_Fatal_Injuries , Total_Serious_Injuries , Total_Minor_Injuries , Total_Uninjured : Evaluate the accident impact.
 - Weather_Condition : Role of weather in accidents.
 - Broad_Phase_Of_Flight : Identifies when accidents occur (e.g., takeoff, landing).
 - Year , Month , Day_Of_Week , Manufacturer_Incident_Count , Model_Incident_Count
- New features for deeper insights.

Less useful features to be dropped:

- Airport_Code : May not be necessary if we have Airport_Name.
- Registration_Number : Unique to each aircraft, not relevant for trend analysis.
- Accident_Number : Identifier that doesn't add analytical value.
- Report_Status & Publication_Date : Administrative data, not directly useful for insights.

```
In [90]: ▶ # Dropping irrelevant columns
columns_to_drop = ['Accident_Number', 'Registration_Number', 'Airport_Code']
aviation_df = aviation_df.drop(columns=columns_to_drop)
```



In [91]: `aviation_df.info()`

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 88678 entries, 0 to 88888
Data columns (total 27 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Event_Id                             88678 non-null  object
1   Investigation_Type                    88678 non-null  object
2   Event_Date                           88678 non-null  datetime64[ns]
3   Location                             88678 non-null  object
4   Country                             88678 non-null  object
5   Airport_Name                         88678 non-null  object
6   Injury_Severity                      88678 non-null  object
7   Aircraft_Damage                      88678 non-null  object
8   Make                                 88678 non-null  object
9   Model                               88678 non-null  object
10  Amateur_Built                        88678 non-null  object
11  Number_Of_Engines                   88678 non-null  float64
12  Engine_Type                         88678 non-null  object
13  Purpose_Of_Flight                   88678 non-null  object
14  Total_Fatal_Injuries                 88678 non-null  float64
15  Total_Serious_Injuries               88678 non-null  float64
16  Total_Minor_Injuries                 88678 non-null  float64
17  Total_Uninjured                     88678 non-null  float64
18  Weather_Condition                    88678 non-null  object
19  Broad_Phase_Of_Flight                88678 non-null  object
20  Year                                 88678 non-null  int64
21  Month                               88678 non-null  int64
22  Day_Of_Week                          88678 non-null  object
23  Manufacturer_Incident_Count          88678 non-null  int64
24  Model_Incident_Count                 88678 non-null  int64
25  Flight_Purpose_Incident_Count          88678 non-null  int64
26  Weather_Incident_Count                88678 non-null  int64
dtypes: datetime64[ns](1), float64(5), int64(6), object(15)
memory usage: 18.9+ MB
```

In [92]: `# Transforming the cleaned data to a csv file`
`aviation_df.to_csv('Aviation_Data_Cleaned.csv', index=False)`

Data Visualization

Aviation safety is a critical aspect of the airline industry, with accidents providing key insights into potential risk factors and areas of improvement. By analyzing historical accident data, we can identify trends, high-risk scenarios, and potential preventive measures. This report presents a series of visualizations that explore patterns in aviation accidents based on aircraft manufacturers, flight phases, and more.

The goal is to uncover meaningful relationships that can help enhance aviation safety, inform industry stakeholders, and provide a data-driven approach to understanding accident dynamics.

Data Overview

The dataset used in this analysis includes aviation accident records spanning multiple decades. The key features analyzed in this report include:

- **Event Date** – To analyze accident trends over time.
- **Aircraft Make & Model** – To assess which manufacturers and models report higher accident occurrences.
- **Phase of Flight** – To determine when accidents are most likely to happen (e.g., takeoff, landing, cruise).
- **Number of Engines & Engine Type** – To analyze the relationship between aircraft configuration and accident rates.
- **Injury Severity & Damage Extent** – To examine the impact of accidents on passengers and aircraft.
- **Purpose of Flight** – To compare accident trends in commercial and private aviation.

By analyzing these features, we aim to highlight key insights that could contribute to risk mitigation and improved aviation policies.

Visualizations and Insights

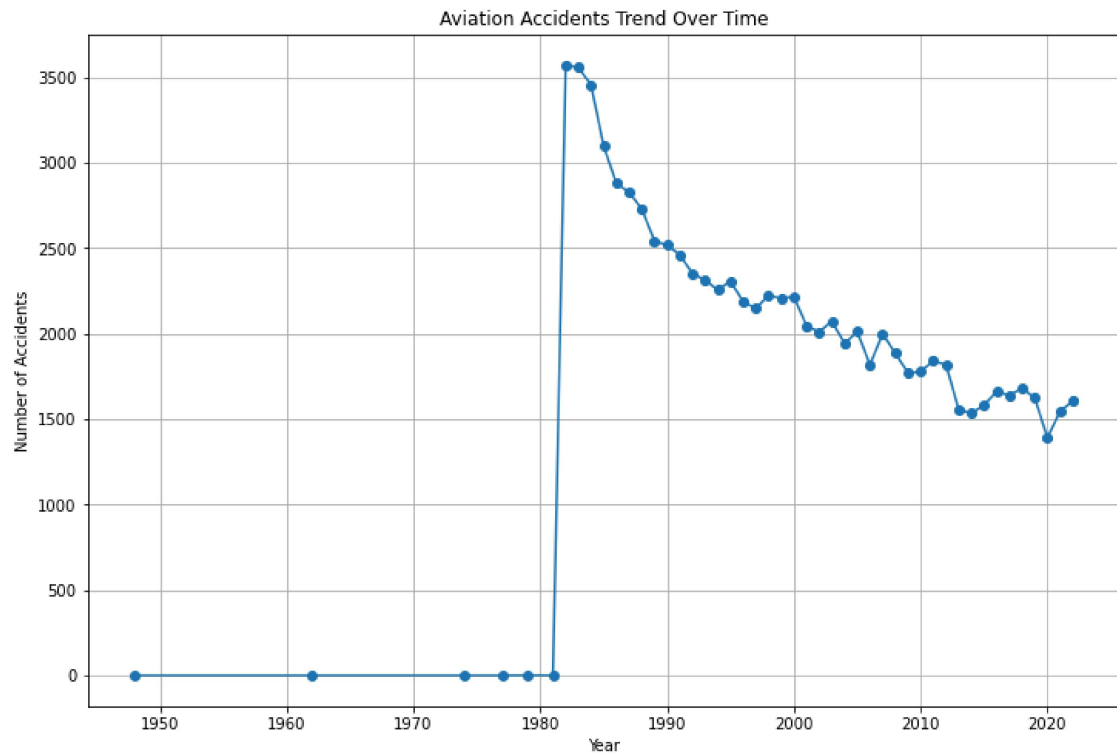
1. Accident Trends Over Time

Understanding how aviation accident rates fluctuate over the years can help identify improvements in safety measures and detect patterns in accident occurrences. A line chart visualizing accident counts per year highlights long-term trends and whether aviation safety has improved over time.

Key Insights:

- Are accident rates declining due to technological advancements and improved regulations?
- Do we see spikes in specific years, possibly linked to policy changes or global events?

```
In [93]: # Accident Trends over Time  
plt.figure(figsize=(12, 8))  
aviation_df.groupby('Year').size().plot(marker='o')  
plt.title('Aviation Accidents Trend Over Time')  
plt.xlabel('Year')  
plt.ylabel('Number of Accidents')  
plt.grid(True)  
plt.show()
```



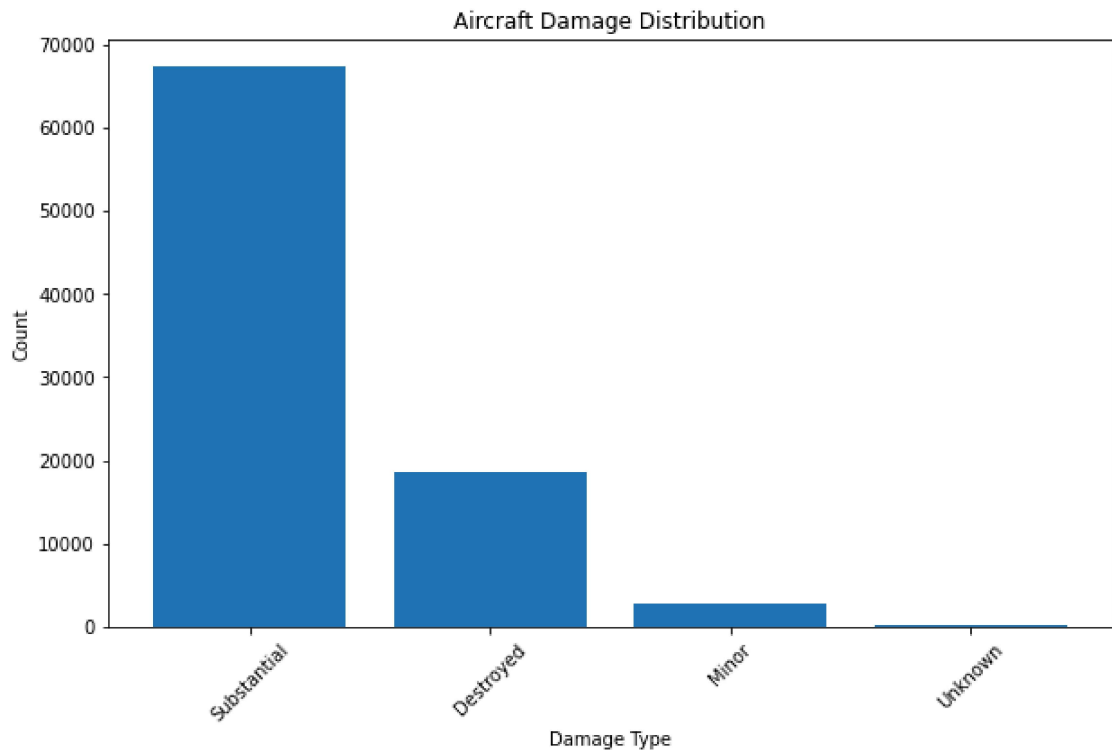
2. Aircraft Damage Types

Different accidents result in varying degrees of aircraft damage. This visualization categorizes incidents based on minor damage, substantial damage, and total loss.

Key Insights:

- What percentage of accidents result in a total aircraft loss?
- Are minor damage incidents more common than catastrophic accidents?
- Can certain aircraft types withstand impact better than others?

```
In [94]: # Aircraft damage types  
# Get counts and sorted order  
damage_counts = aviation_df['Aircraft_Damage'].value_counts()  
categories = damage_counts.index  
  
# Create figure  
plt.figure(figsize=(10, 6))  
plt.bar(categories, damage_counts)  
  
# Add labels and title  
plt.title('Aircraft Damage Distribution')  
plt.xlabel('Damage Type')  
plt.ylabel('Count')  
  
# Rotate x-axis labels  
plt.xticks(rotation=45)  
  
# Show plot  
plt.show()
```



3. Accidents by Phase of Flight

Not all phases of flight carry the same level of risk. By analyzing which phase—takeoff, climb, cruise, descent, or landing—has the highest accident occurrences, we can pinpoint critical moments that require extra precaution.

Key Insights:

- Do most accidents occur during takeoff and landing, the most complex phases of flight?
- Are mid-flight accidents less common, or do certain conditions make them more frequent?

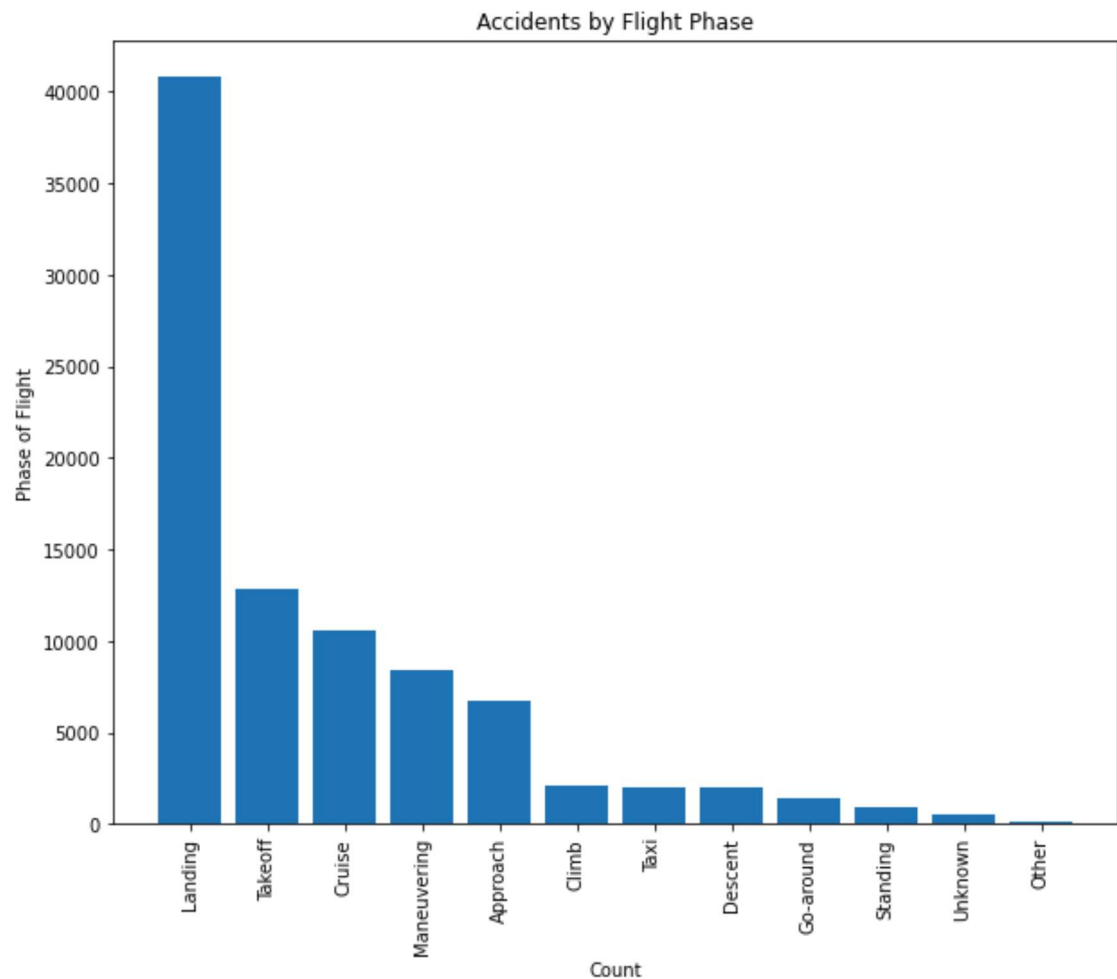
- How can airlines improve safety procedures in high-risk phases?

```
In [95]: ▶ # Get counts and sorted order
flight_phase_counts = aviation_df['Broad_Phase_Of_Flight'].value_counts()
categories = flight_phase_counts.index

# Create figure
plt.figure(figsize=(10, 8))
plt.bar(categories, flight_phase_counts)

# Add Labels and title
plt.title('Accidents by Flight Phase')
plt.xlabel('Count')
plt.ylabel('Phase of Flight')

# Rotate x-axis labels
plt.xticks(rotation=90)
# Show plot
plt.show()
```



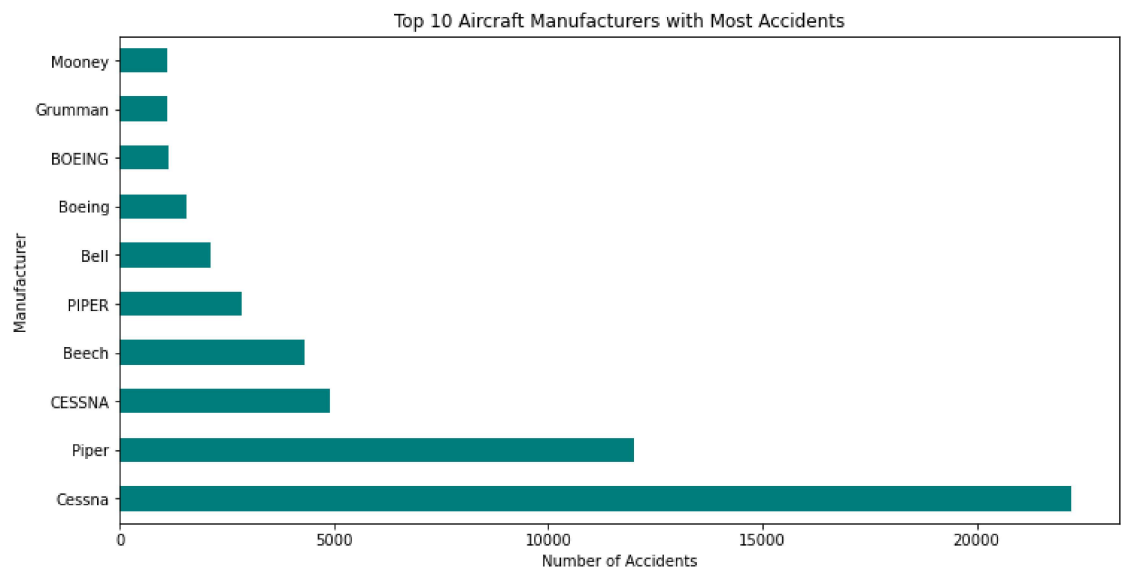
4. Top Manufacturers Involved in Accidents

Some aircraft manufacturers produce a higher volume of planes, naturally leading to more incidents. This visualization compares accident frequencies among major aircraft manufacturers.

Key Insights:

- Do certain manufacturers have disproportionately high accident rates?

```
In [96]: ▶ # Top manufacturers involved in accidents
plt.figure(figsize=(12, 6))
aviation_df['Make'].value_counts().nlargest(10).plot(kind='barh', color='teal')
plt.title('Top 10 Aircraft Manufacturers with Most Accidents')
plt.xlabel('Number of Accidents')
plt.ylabel('Manufacturer')
plt.show()
```



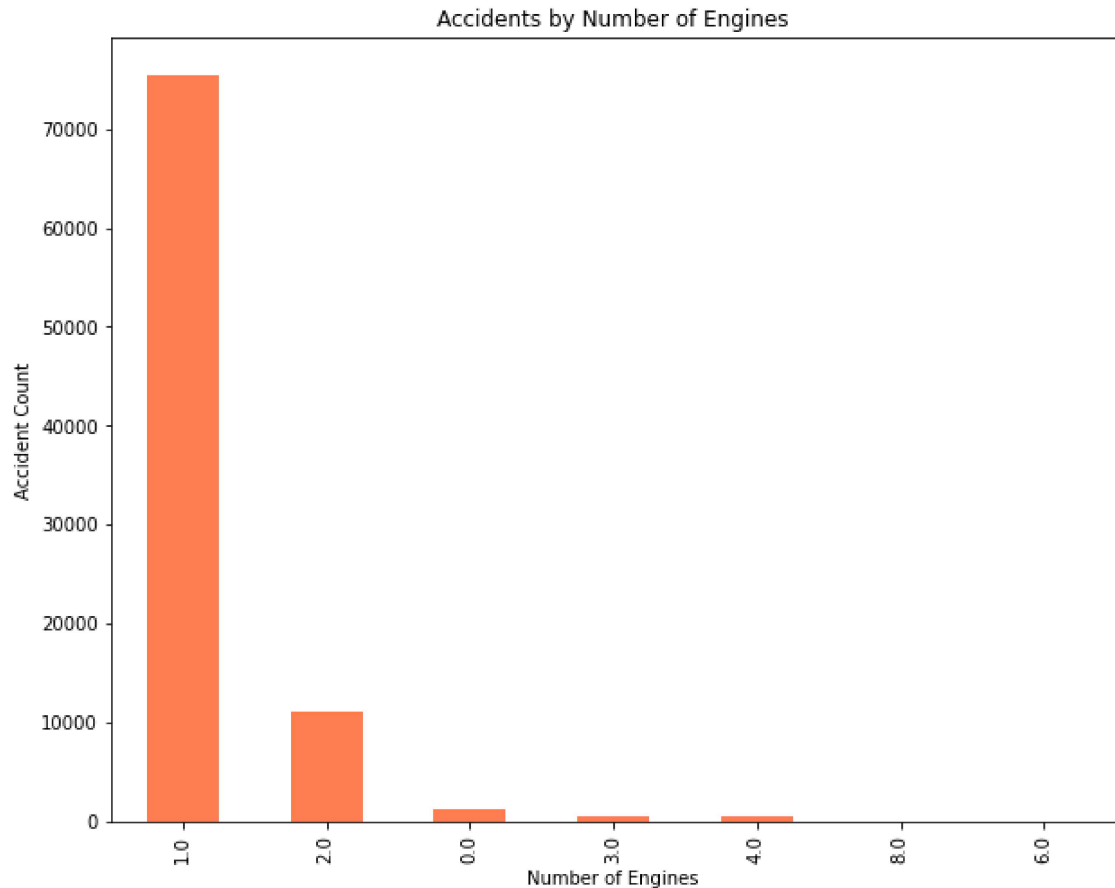
5. Number of Engines vs. Accidents

This analysis explores whether aircraft with more engines experience fewer or more accidents. We compare single-engine, twin-engine, and multi-engine aircraft.

Key Insights:

- Do single-engine aircraft have higher accident rates due to mechanical failure risks?
- Are multi-engine aircraft safer, or do complex systems introduce more failure points?
- How does engine count influence accident survival rates?

```
In [97]: ▶ # Number of engines vs. accidents
plt.figure(figsize=(10, 8))
aviation_df['Number_Of_Engines'].value_counts().plot(kind='bar', color='coral')
plt.title('Accidents by Number of Engines')
plt.xlabel('Number of Engines')
plt.ylabel('Accident Count')
plt.show()
```



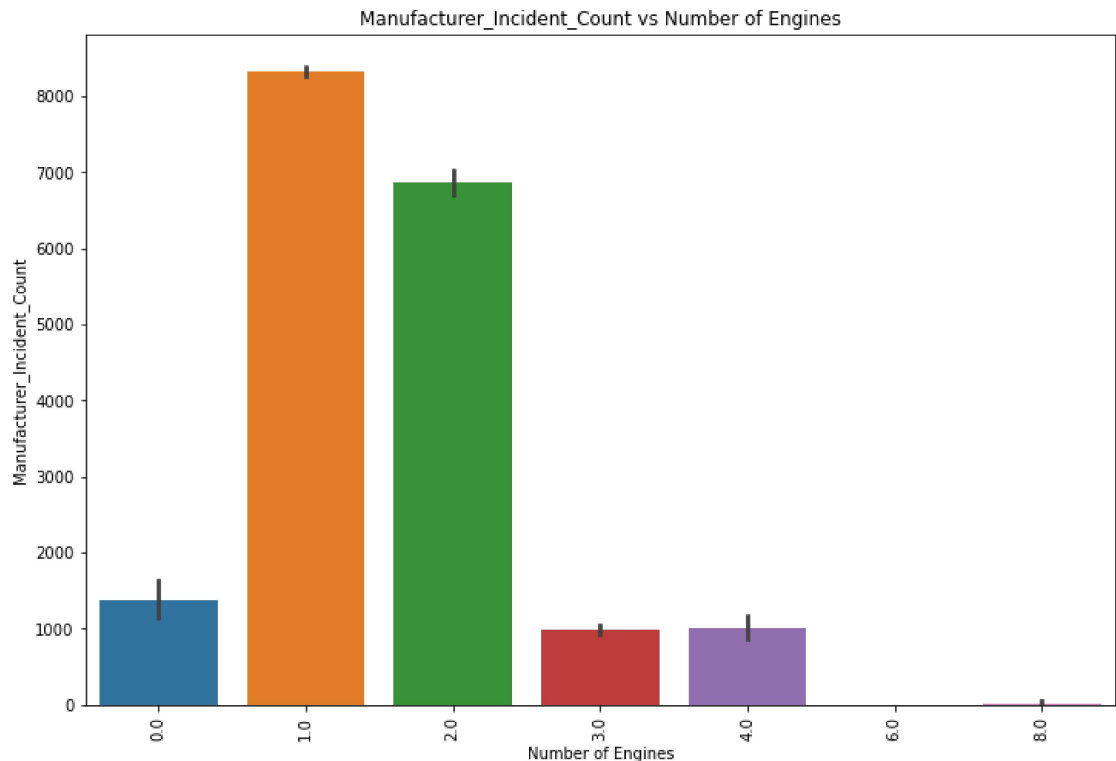
6. Manufacturer vs. Number of Engines

Different aircraft manufacturers design planes with varying engine configurations. A box plot of manufacturers and engine counts provides insights into their aircraft designs.

Key Insights:

- Do some manufacturers favor single-engine designs, while others focus on multi-engine planes?
- How do engine configurations impact accident trends for different manufacturers?
- Are manufacturers producing a diverse range of aircraft, or specializing in specific types?

```
In [98]: ▶ # Manufacturer vs Number of Engines
plt.figure(figsize=(12, 8))
sns.barplot(x=aviation_df['Number_Of_Engines'], y=aviation_df['Manufacturer_Incident_Count'])
plt.xticks(rotation=90)
plt.title('Manufacturer_Incident_Count vs Number of Engines')
plt.xlabel('Number of Engines')
plt.ylabel('Manufacturer_Incident_Count')
plt.show()
```



7. Purpose of Flight vs. Injury Severity

Different flight purposes, such as commercial, private, cargo, and military, may have varying levels of accident severity. This visualization examines the **average number of injuries** for each flight purpose to understand how risk levels differ across categories.

Key Insights:

- Do certain flight purposes experience higher fatal or serious injuries?
- Are private flights more prone to severe injuries compared to commercial flights?
- Which flight categories tend to have the highest number of uninjured passengers?

```

In [104]: # Define the injury types
injury_types = ['Total_Fatal_Injuries', 'Total_Serious_Injuries', 'Total_M

# Group by Purpose of Flight and compute mean injuries
purpose_injuries = aviation_df.groupby('Purpose_Of_Flight')[injury_types].

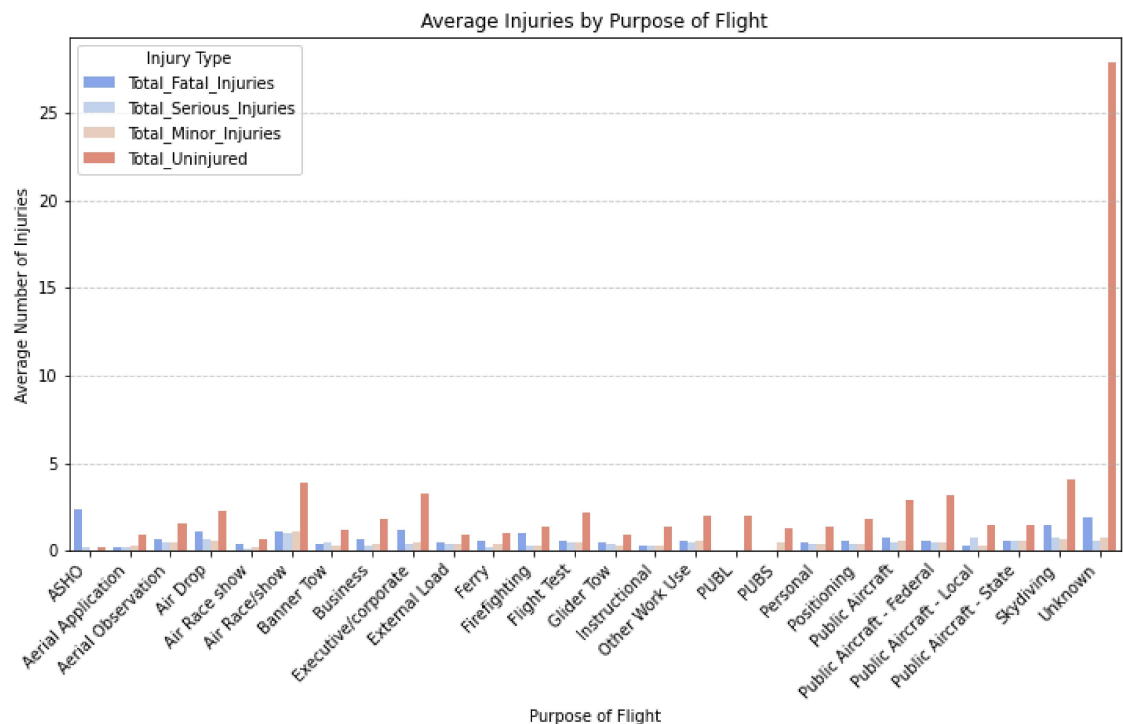
# Melt the dataframe for easier plotting
purpose_injuries_melted = purpose_injuries.melt(id_vars=['Purpose_Of_Flight',
                                                         var_name='Injury_Type',
                                                         value_name='Average_Injur

# Set up the figure
plt.figure(figsize=(12, 6))

# Plot grouped bar chart
sns.barplot(
    data=purpose_injuries_melted,
    x='Purpose_Of_Flight',
    y='Average_Injuries',
    hue='Injury_Type',
    palette='coolwarm'
)

# Customize plot
plt.xticks(rotation=45, ha='right') # Rotate labels for readability
plt.xlabel("Purpose of Flight")
plt.ylabel("Average Number of Injuries")
plt.title("Average Injuries by Purpose of Flight")
plt.legend(title="Injury Type")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

```



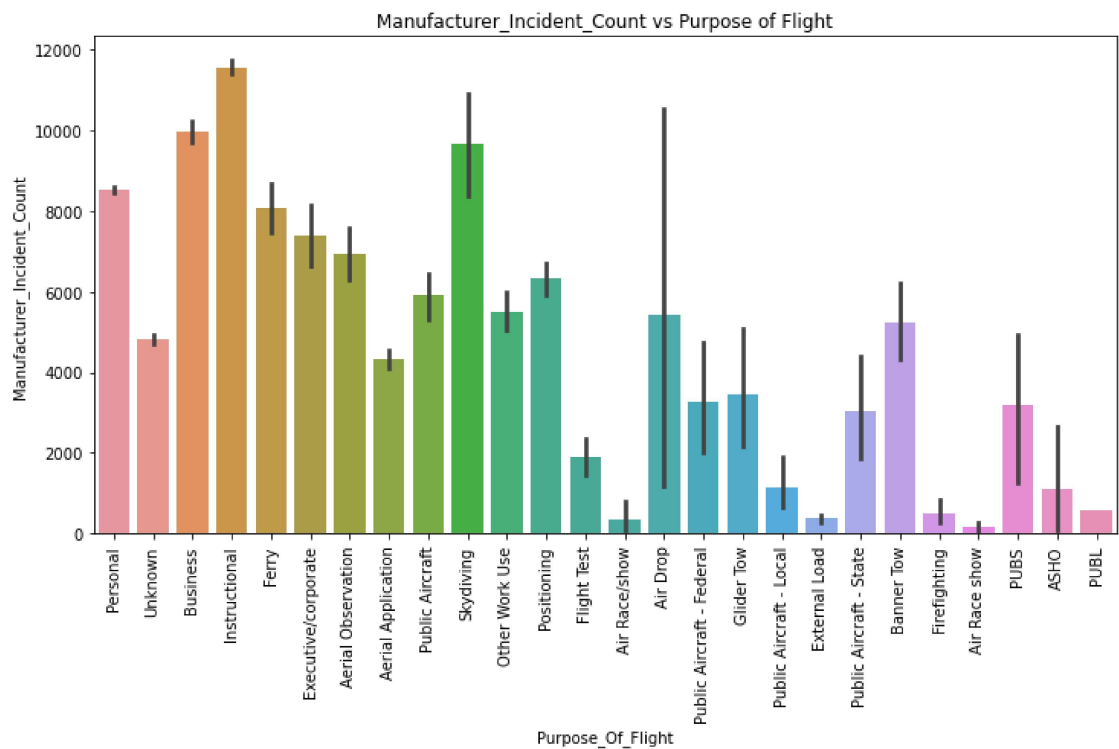
8. Manufacturer vs. Purpose of Flight

Different aircraft are built for different purposes, such as commercial transport, private aviation, cargo, and military use. This visualization compares manufacturers based on the primary purpose of their aircraft.

Key Insights:

- Are some manufacturers more dominant in commercial aviation, while others focus on private or military aircraft?
- Does the purpose of flight influence accident frequency (e.g., are private flights riskier than commercial ones)?
- How do cargo flights compare to passenger flights in terms of safety?

```
In [100]: # Manufacturer vs Purpose of Flight
plt.figure(figsize=(12, 6))
sns.barplot(x=aviation_df['Purpose_Of_Flight'], y=aviation_df['Manufacturer_Incident_Count'])
plt.xticks(rotation=90)
plt.title('Manufacturer_Incident_Count vs Purpose of Flight')
plt.xlabel('Purpose_Of_Flight')
plt.ylabel('Manufacturer_Incident_Count')
plt.show()
```



9. Correlation Heatmap of Injury Severity and Number of Engines

Understanding the relationship between injury severity and aircraft characteristics can provide insights into accident outcomes. This heatmap visualizes the correlation between different injury severity levels and the number of engines in aviation accidents.

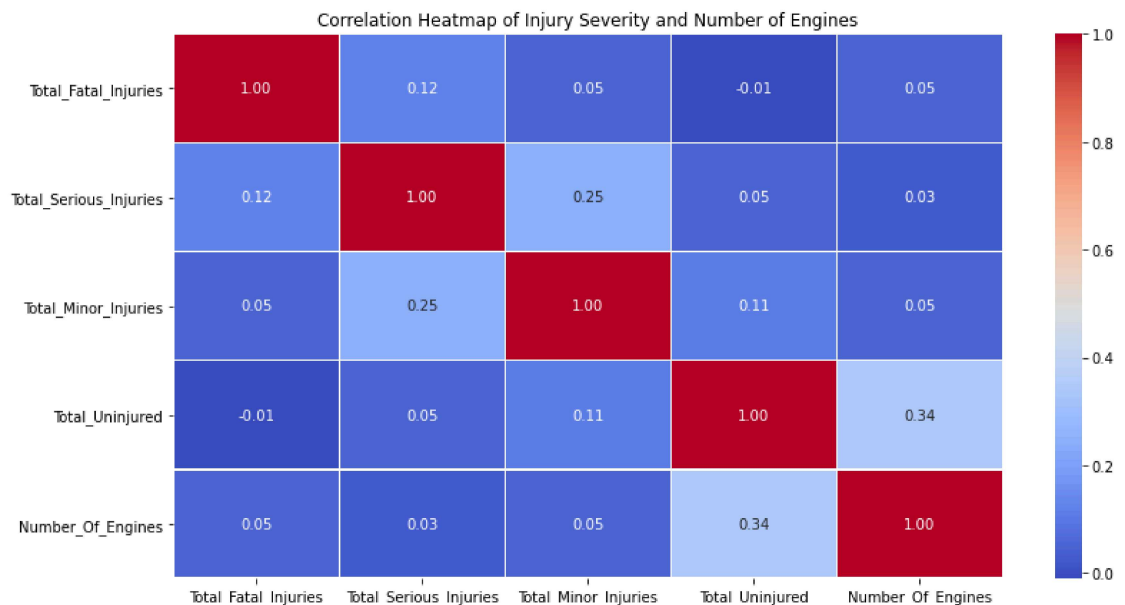
Key Insights:

- Are aircraft with more engines associated with higher or lower injury severity?
- Do fatal, serious, minor, and uninjured cases show strong correlations with each other?
- How does the number of engines relate to injury severity patterns in aviation accidents?

```
In [101]: # Select numerical columns for correlation
num_cols = ['Total_Fatal_Injuries', 'Total_Serious_Injuries', 'Total_Minor_Injuries', 'Total_Uninjured', 'Number_Of_Engines']

# Compute correlation matrix
correlation_matrix = aviation_df[num_cols].corr()

# Plot heatmap
plt.figure(figsize=(13, 7))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f", li
plt.title("Correlation Heatmap of Injury Severity and Number of Engines")
plt.show()
```



Conclusion

These visualizations offer valuable insights into aviation accident trends, helping to identify high-risk scenarios, critical safety concerns, and potential preventive measures. By leveraging data analytics, aviation regulators, manufacturers, and airlines can work towards enhancing flight safety, reducing accident risks, and improving overall industry standards.

Further analysis could explore additional factors such as pilot experience, maintenance history, and specific accident causes to gain a more comprehensive understanding of aviation safety.

